

Assessment 3: Project

Investigating Keystroke Dynamics for User Authentication

Lim Dong Xian (14586049)

Bachelor of Information Technology, James Cook University

CP3416: Behavioural Cybersecurity

Mr. Roberto Dillon

7 April 2026

Table of Contents

Abstract	3
1. Introduction.....	4
2. Tools Used	4
2.1 Programming Tools	4
2.2 Version Control (GitHub).....	5
2.3 Machine Learning Platform	5
2.4 Libraries Used.....	6
2.4.1 Python library.....	6
2.4.2 Kaggle library	7
3. Keylogger Program Implementation.....	8
3.1 Program Workflow	8
3.2 Keystroke Logging Program.....	10
3.3 Data Collection	10
3.4 Dataset Creation.....	11
4. Machine Learning Implementation.....	12
4.1 Baseline Models.....	12
4.2 Model Tuning.....	13
4.3 Final Model.....	15
5. Results Analysis.....	17
5.1 Overall Model Performance.....	17
5.2 Analysis of Keystroke Features	17
5.3 Effect of Data Quality and Noise.....	18
5.4 Interpretation of the Results.....	18
5.5 Additional Feature Filtering Experiments	18
6. Problems Encountered	19
6.1 Keylogger Compatibility Issue	19
6.2 User Fatigue During Data Collection	19
6.3 Executable File Blocked by System Security	19
7. Testing and Quality Assurance	20
7.1 Code and Data Collection Testing	20
7.2 Baseline Model Testing	20
7.3 Model Testing Using Cross-Validation	21

7.4 Performance Evaluation and Quality Assurance	21
7.5 Verification of Improvement from Baseline to Tuned Model	21
8. Limitation.....	21
8.1 Limited Number of Participants and Data Size	21
8.2 Use of Fixed Sentences	22
8.3 Limited Feature Scope	22
8.4 Platform and Data Collection Constraints	22
8.5 Limitations of the Machine Learning Model	23
Conclusion	23
Reference	24
Sound effect	24
Appendix:.....	24
Python code:.....	24
Kaggle (Notebook):	25
Kaggle (Data):.....	25

Abstract

This project studies the use of keystroke dynamics as a behavioural biometric for user authentication. A dynamic keylogging program was developed in Python with a graphical interface to collect detailed typing data, including hold time, flight time, and digraph patterns. Data from both an authenticated user and intruders were used to train and evaluate several machine learning models, with the final result using Random Forest achieving the highest accuracy of 77.05% and an AUC of 0.843 after hyperparameter tuning. The study demonstrates that combining multiple keystroke features can effectively distinguish users, highlighting the potential of behavioural biometrics for secure authentication systems, while also acknowledging limitations in dataset size and sentence diversity.

1. Introduction

User authentication is a critical component of modern cybersecurity, traditionally relying on knowledge-based methods such as passwords or PINs. Keystroke dynamics has emerged as an important behavioural biometric for cybersecurity, with recent surveys documenting significant advances in authentication algorithms and datasets (Wahab et al., 2023). However, these methods can be compromised through theft, phishing, or brute-force attacks. Behavioural biometrics, such as keystroke dynamics, offer a promising alternative by analysing an individual's typing patterns to verify identity. This project explores the development of a dynamic keylogging system that captures keystroke timing features in a controlled environment and applies machine learning techniques to classify users as authenticated or intruders. By combining time characteristics features such as hold time and flight time with key transition patterns (digraphs), the study aims to demonstrate the effectiveness of keystroke dynamics as a secure, non-intrusive method of authentication, while addressing practical challenges in data collection and model implementation.

2. Tools Used

2.1 Programming Tools

The dynamic keylogging program was created using the Python 3 programming language. Python was selected in this project as it is a popular language and easy to implement, especially for projects that involve simple graphical user interfaces, data processing and machine learning.

In addition, Python has a wide range of libraries that make it suitable for capturing keystroke events and processing timing information accurately. These helped improve development efficiency and made it easier to organise, especially when integrating multiple components such as the graphical user interface, sound effects, and data logging functions.

Python was also helpful in storing generated keystroke data into CSV format and later used directly in machine learning platforms such as Kaggle. This allowed the development process to remain consistent from data collection to model training.

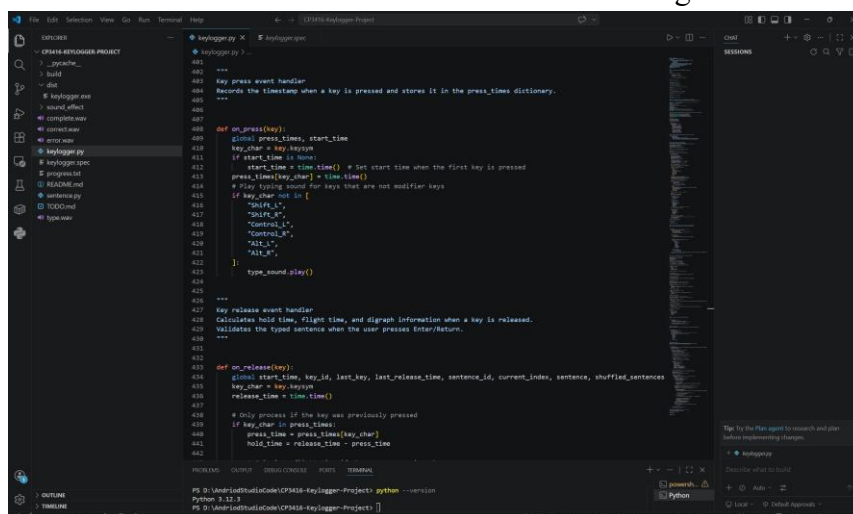


Figure 1: Simple overview of the Python code

2.2 Version Control (GitHub)

Version control was used to manage the development process of the project. The source code was stored and maintained using GitHub, which allowed different versions of the program to be tracked and updated. Separated branches were created to test new features, such as the sound effects, and attempts to extract the program into a single executable file, before merging them into the main version.

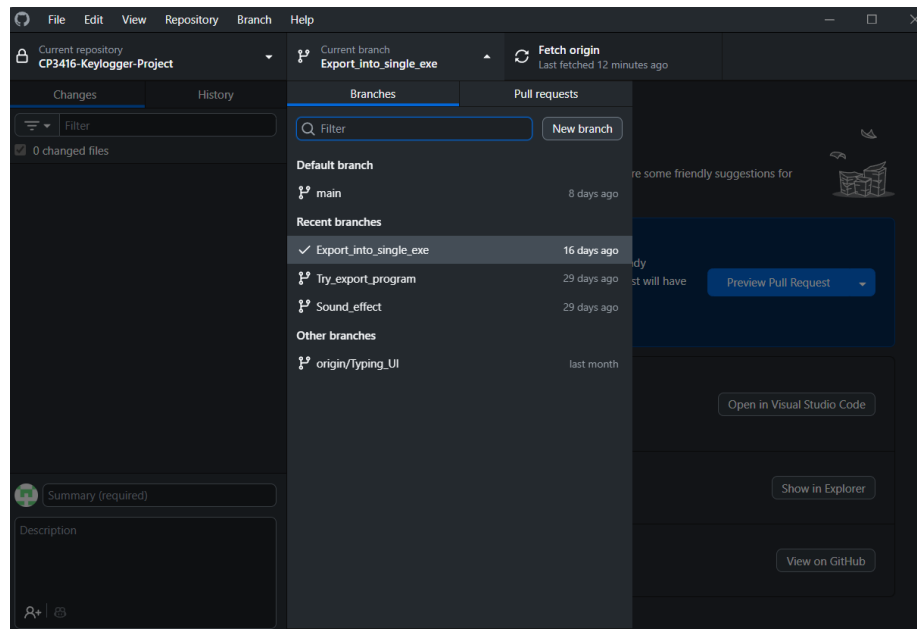


Figure 2: Demonstrate the GitHub version control

2.3 Machine Learning Platform

After the keystroke data was collected using the key logger program, the generated CSV file was uploaded to the Kaggle database. The dataset was then imported into a Kaggle Notebook, where the data loading, data preprocessing, model training, and result comparison were carried out.

The machine learning models in this project were developed and tested using Kaggle Notebook. Kaggle was selected as the machine learning platform because it provides a cloud-based environment that allows users to train and evaluate machine learning models without requiring high-performance local hardware. This is especially useful for projects that involve dataset analysis, multiple runs with model parameter comparison, and performance evaluation. Kaggle provides built-in support for Python and commonly used machine learning and data analysis libraries, which makes it suitable for implementing different classification models in this project. Kaggle also support version saving, which allows easy storing of experimentation with multiple models and training settings, then comparing with the result later on.

In this project, several baseline models were tested and compared before selecting the Random Forest algorithm, which returned the highest overall performance as the final model. This model was then carried out multiple tests and tuning to analyse model performance, including evaluation metrics such as accuracy, precision, recall, F1-score, ROC curve and AUC result.

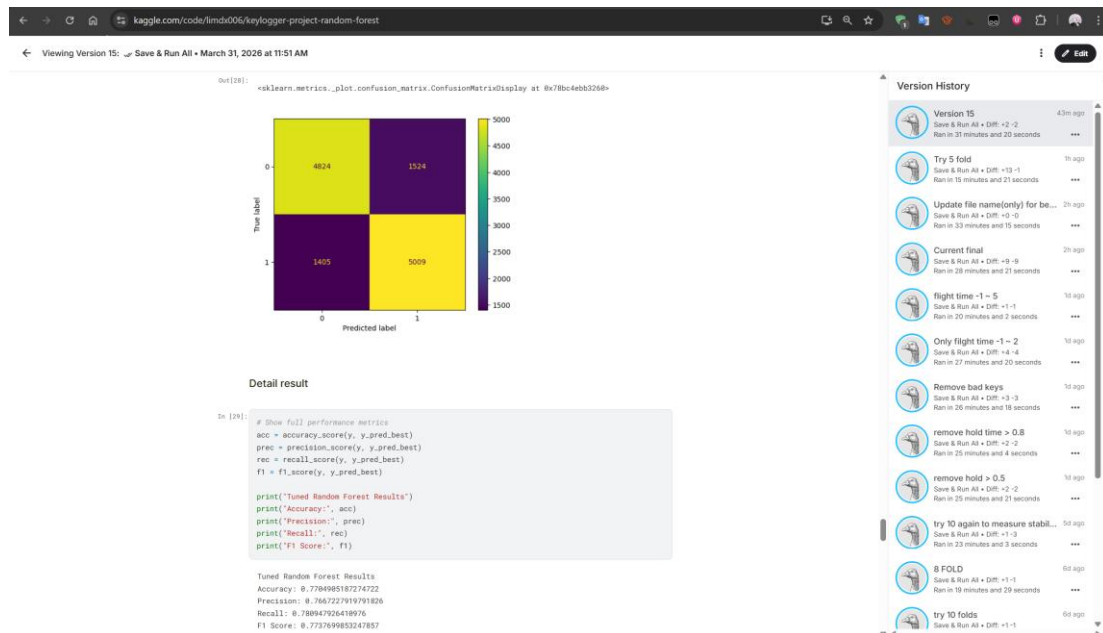


Figure 3: Kaggle version selection page and a sample of the results

2.4 Libraries Used

2.4.1 Python library

The dynamic keylogging program was created using several Python libraries to support the graphical user interface, keystroke data collection, timing measurement, and dataset generation. Each library was used based on its specific functionality required for the project. The following libraries are used:

- The **Tkinter** library was used to develop the simple graphical user interface (GUI) of the keylogger program.
 - It was used to create the consent dialogue box, the typing window, the text input field, labels, and control buttons like the “Save & Exit” button.
 - It was also used to capture keyboard events through Tkinter's built-in event binding system, such as <KeyPress> and <KeyRelease> event detection.
 - Tkinter was selected because it is lightweight, easy to use and already included in the Python standard library, making it suitable for developing an interactive visual keylogging application.
- The **Time** library was used to record timestamps when each key was pressed and released. This allowed the program to calculate important keystroke dynamics features such as hold time and flight time, which are essential for analysing typing behaviour.
- The **UUID** library was used to generate a unique user ID for each participant. This ensures that the collected data can be organised and keeps the participants' information anonymous, such as their names.
- The **random** library was used to randomly select and shuffle sentences for the typing test. This helps reduce typing bias and ensures that the collected data are more representative of natural typing behaviour.

- The **CSV** library was used to store the collected keystroke data into a structured CSV file. This file format is suitable for machine learning because it allows the data to be easily imported into platforms such as Kaggle to carry out data analysis.
- The **Pygame** library was used to add sound effects during the typing section. Sound effects were played during the typing section to reduce boredom.
- The **SYS and OS** libraries were used to manage file paths to ensure that the program can run correctly when exported as a single executable file.

2.4.2 Kaggle library

Several Python libraries were used in the Kaggle Notebook to perform data preprocessing, model training, and performance evaluation. These libraries were selected because they are widely used in machine learning projects and provide reliable tools for classification tasks. The following libraries are used:

- The **pandas** library was used to load and process the keystroke dataset stored in CSV format. It was used to clean the data, organise the features, and prepare the dataset before training the models.
- The **os** library was used to manage file paths and directories within the Kaggle environment. Since the keystroke dataset was uploaded as multiple CSV files, it helped the program locate and access the dataset files correctly.
- The **glob** library was used to search and load multiple CSV files automatically. This made it easier to combine multiple keystroke datasets into a single dataset for machine learning.
- The **scikit-learn(sklearn)** library was the main library used for machine learning. It was used to implement several classification models, including K-Nearest Neighbour, Logistic Regression, Support Vector Machine, Gradient Boosting and Random Forest.
 - The **StandardScaler** and **LabelEncoder** functions from scikit-learn were used for data preprocessing. These tools helped convert categorical data into numerical values and normalise the dataset before training the models.
 - The **GridSearchCV** and **StratifiedKFold** tools were used for model tuning and cross-validation. These techniques helped improve the performance of the Random Forest model and ensured that the evaluation results were reliable.
 - The **roc_curve** and **auc** functions were used to generate the Receiver Operating Characteristic (ROC) curve and calculate the Area Under the Curve (AUC). This helped analyse the trade-off between true positive and false positive rates for the final Random Forest model and provided a robust measure of its predictive performance.
 - The **pipeline** class was used to streamline the machine learning workflow by chaining preprocessing steps and the model training into a single object.
 - The metrics functions including **accuracy_score**, **precision_score**, **recall_score** and **f1_score** were used to evaluate the performance of the models. These metrics provide a comprehensive assessment of classification quality, measuring not only overall accuracy but also the model's ability to correctly identify each class.

- The **confusion_matrix** and **ConfusionMatrixDisplay** tools were used to visualise the performance of the classification models, allowing for the visual identification of misclassified samples.
- The **matplotlib** library was used to visualise the data and results of the machine learning models. It was used to display the distribution of the data on the data analysis section, confusion matrices, and the ROC curve on the final result section.

3. Keylogger Program Implementation

3.1 Program Workflow

The keylogger program was created as a structured, dynamic keylogging program. The workflow of the program is designed to guide the user through a controlled typing test while recording participants' keystroke behaviour and generating a dataset for machine learning analysis.

The program starts with creating a unique user ID using the UUID library. This ensures that each participant's data is stored separately and can be clearly identified during the dataset construction stage. The program will then display a window with a consent dialogue box that explains the purpose of the study and asks the user for permission to participate. The program will continue if the user clicks on the check box of acknowledgement and agrees to participate. If the user does not agree, the program will exit immediately.

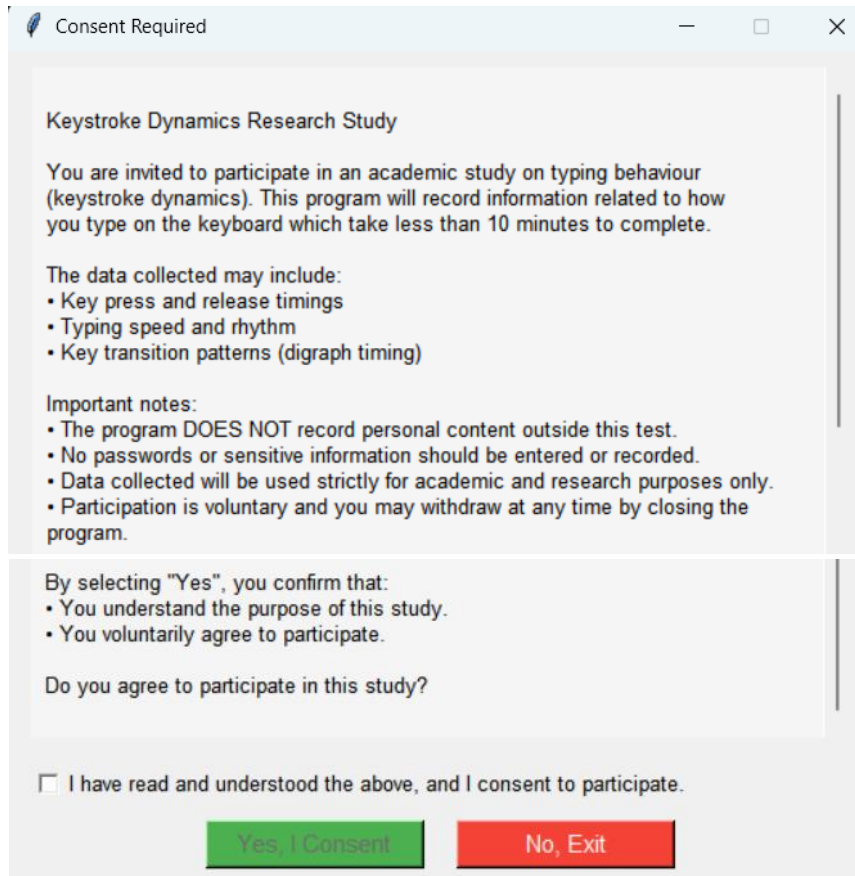


Figure 4 & 5: Full consent message window

The program was developed mainly using the tkinter library, which ensures that all keystroke recording only takes place within the GUI of the application. The system does not record any keystrokes outside the typing window and does not run in the background. Keystroke data is only captured when the user actively types in the provided text input field during the typing test. This design ensures that the program functions as a controlled data collection tool for research purposes only.

After agreement, a small reminder window will show up to remind participants that this is a study of natural typing, not a competition of typing speed. Once consent and the reminder are given, the typing test interface is displayed. The program will randomly select a sentence from a predefined list and display it on the typing window above the user input text box, then the participants are required to type the sentence exactly as shown in the text input field (case sensitive and punctuation marks). This process is repeated until at least 20 sentences are correctly typed.

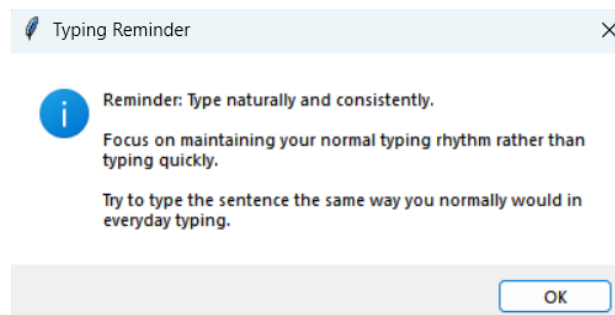


Figure 6: Reminder pop-up window

While the user is typing, the program records detailed keystroke information in real time. For every key press and release, the program captures and records the timestamp, then calculates important keystroke dynamics features such as hold time and flight time. These features are stored temporarily in memory until the typing session is completed.

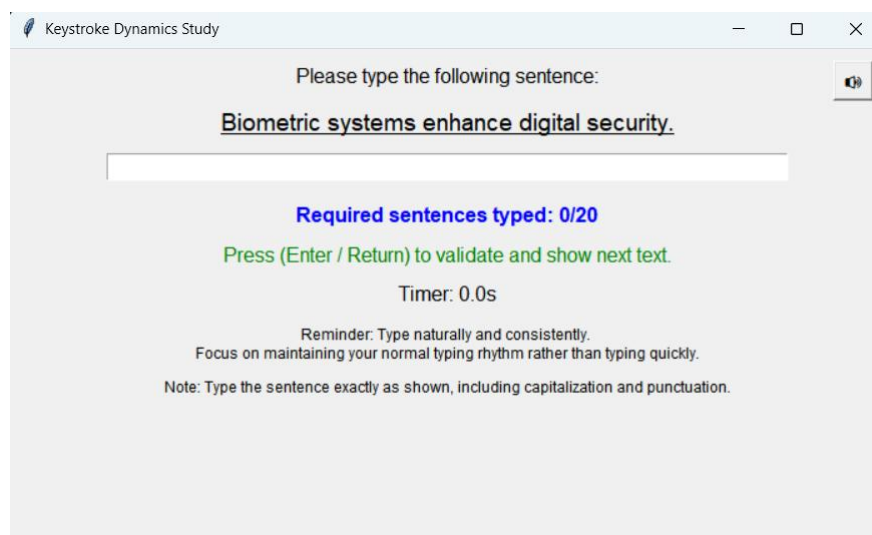


Figure 7: Initial typing window

The user is only allowed to save the recorded data when the required number of sentences has been met. The program will convert the collected keystroke information into a structured CSV file, which contains all necessary features for future machine learning and

data analysis usage. Noted that the participants can choose the location to save and will need to send the CSV file they saved back to me manually.

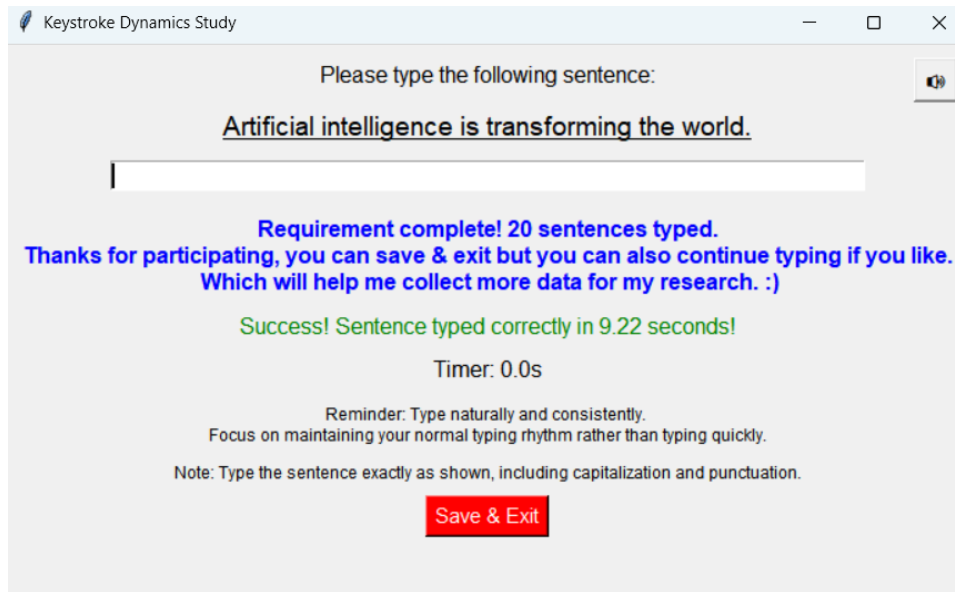


Figure 8: When the requirement is reached, the save button will appear.

3.2 Keystroke Logging Program

The keystroke logging feature is the main component of this program, as it records detailed typing behaviour data for machine learning analysis. Instead of only recording the text typed by the user, the program mainly captures keystroke dynamics such as hold time, flight time, and digraphs.

The program detects both key-press and key-release events using the tkinter built-in feature. When a key is pressed, and when the key is released, the timestamp of each is recorded, and then the program uses the two timestamps to calculate the hold time. The program also calculates the flight time, which represents the time between releasing one key and pressing the next key. In some cases, the flight time might appear as a negative value because a user may press the next key before fully releasing the previous key, especially when users are typing an upper-case letter by holding the shift key. This value is kept as it is a common natural typing which helps reflect the user's real typing rhythm more accurately.

The program also records digraph information, which represents the relationship between two consecutive keys. Since all participants type the same set of predefined sentences repeatedly, the program is able to collect consistent digraph patterns instead of random key combinations. This makes the dataset more suitable for machine learning, as the same digraphs appear in multiple samples, allowing the model to compare typing behaviour more accurately.

3.3 Data Collection

The dataset used in this project was collected using the typing interface developed in Python. There are two sets of data to be collected, including the keystroke data of authenticated users and intruder users. The total sentence of each should also be the same to ensure the balance of the label.

For the authenticated user, the typing data was recorded by the developer (me myself) across multiple days at different times, instead of in a single session. This was done to avoid memorising the sentences, which could affect typing behaviour and reduce the reliability of the dataset. Approximately 120 sentences were collected for the authenticated user.

To simulate intruder behaviour, the typing program was shared with several friends who were asked to type the same set of sentences. Around 120 sentences were also collected from intruder users.

The same set of sentences was used for both groups to ensure consistency of the digraph in the dataset and allow accurate comparison of typing patterns.

3.4 Dataset Creation

The keystroke data collected by the program is converted into a structured dataset that can be used for machine learning. During the typing session, all keystroke information is temporarily stored in memory. Each key event is recorded together with important attributes, including sentence ID, key value, key order, digraph, press timestamp, release timestamp, hold time, and flight time.

Each row in the dataset represents a single key event, while each column represents a specific feature. Because all participants type the same predefined sentences, the dataset remains consistent and suitable for training and evaluating the machine learning models in the next part of the project.

	A	B	C	D	E	F	G	H	I
1	user_id	sentence_i	key	key_id	digraph	press_ts	release_ts	hold_time	flight_time
2	U_4a33b8	1	Shift_L	1	None	1.77E+09	1.77E+09	0.2561	0
3	U_4a33b8	1	r	2	Shift_L_r	1.77E+09	1.77E+09	0.1318	0.1236
4	U_4a33b8	1	i	3	r_i	1.77E+09	1.77E+09	0.1377	0.0361
5	U_4a33b8	1	t	4	i_t	1.77E+09	1.77E+09	0.1076	0.2042
6	U_4a33b8	1	BackSpace	5	t_BackSpa	1.77E+09	1.77E+09	0.0434	0.2868
7	U_4a33b8	1	BackSpace	6	BackSpace	1.77E+09	1.77E+09	0.0531	0.0622
8	U_4a33b8	1	t	7	BackSpace	1.77E+09	1.77E+09	0.1115	0.1214
9	U_4a33b8	1	i	8	t_i	1.77E+09	1.77E+09	0.0994	0.0161
10	U_4a33b8	1	f	9	i_f	1.77E+09	1.77E+09	0.145	0.2967
11	U_4a33b8	1	i	10	f_i	1.77E+09	1.77E+09	0.1114	0
12	U_4a33b8	1	c	11	i_c	1.77E+09	1.77E+09	0.123	0.1052
13	U_4a33b8	1	i	12	c_i	1.77E+09	1.77E+09	0.1209	-0.002
14	U_4a33b8	1	a	13	i_a	1.77E+09	1.77E+09	0.129	0.0121
15	U_4a33b8	1	l	14	a_l	1.77E+09	1.77E+09	0.0984	0
16	U_4a33b8	1	space	15	l_space	1.77E+09	1.77E+09	0.093	0.1378
17	U_4a33b8	1	i	16	space_i	1.77E+09	1.77E+09	0.1027	0.225
18	U_4a33b8	1	n	17	i_n	1.77E+09	1.77E+09	0.1164	0.034
19	U_4a33b8	1	t	18	n_t	1.77E+09	1.77E+09	0.179	0.0357
20	U_4a33b8	1	e	19	t_e	1.77E+09	1.77E+09	0.1527	-0.0441
21	U_4a33b8	1	l	20	e_l	1.77E+09	1.77E+09	0.0887	-0.0127
22	U_4a33b8	1	l	21	l_l	1.77E+09	1.77E+09	0.0605	0.1986
23	U_4a33b8	1	i	22	l_i	1.77E+09	1.77E+09	0.0711	0.1016
24	U_4a33b8	1	g	23	i_g	1.77E+09	1.77E+09	0.1531	0.1496
25	U_4a33b8	1	e	24	g_e	1.77E+09	1.77E+09	0.1369	-0.046

Figure 9: An example of the row and column of the CSV file

4. Machine Learning Implementation

4.1 Baseline Models

The baseline models were developed progressively through several versions of the machine learning workflow. The first few versions followed a simple and standard machine learning pipeline that included loading the dataset, combining the files, applying label encoding, splitting the dataset into 80% training data and 20% testing data, training the model and finally evaluating each model's performance.

The three key features used include hold time, flight time, and encoded digraph, which were selected based on their demonstrated importance in keystroke dynamics literature. Hold time and flight time represent fundamental temporal characteristics of typing rhythm, while the encoded digraph captures the sequential relationship between consecutive keystrokes. This combination aligns with established keystroke authentication research that identifies temporal features and digraph patterns as the most discriminative behavioural biometric indicators (Wahab et al., 2023).

After the initial version was completed, additional classification models were introduced in order to compare performance across different algorithms. The models implemented in the baseline stage include:

- K-Nearest Neighbour (KNN)
- Logistic Regression
- Random Forest
- Support Vector Machine (SVM)
- Gradient Boosting

Further improvements were then made by analysing the dataset. The distribution of hold time and flight time was visualised, and the number of unique digraphs was also studied. Also, a detail of the hold time, flight time is printed out using the `describe()` feature.

```
File: U_6cde40_log.csv
Shape: (1628, 9)
   hold_time  flight_time
count  1628.000000  1628.000000
mean    0.105068    0.099651
std     0.053618    0.189762
min     0.000000   -0.815000
25%     0.077900    0.012400
50%     0.096700    0.064850
75%     0.122025    0.133950
max     0.842100    2.635900
```

Figure 10: Hold and flight time describe

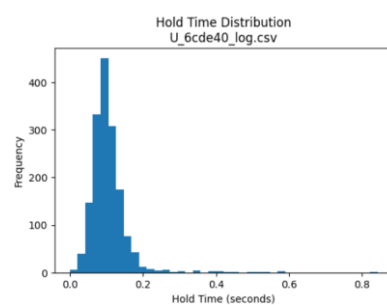


Figure 11: Hold time distribution graph

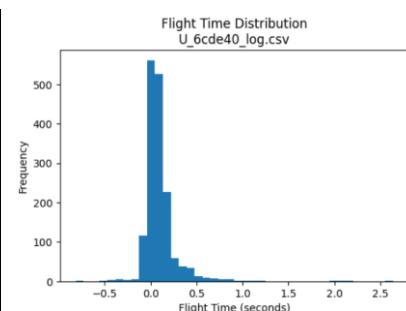


Figure 12: Flight time distribution graph

Based on this analysis, the StandardScaler was added to the workflow because some models, especially SVM, perform better when the data is standardised (Oliveira et al., 2025). Additionally, the digraph feature was encoded using LabelEncoder to convert categorical key-pair strings into numerical values, as machine learning models require numerical input features (Wahab et al., 2023). Some attempt was also made to remove “noise” based on the study of the mean and the 75th percentile, but this method reduced the performance of most of

the models, which might indicate that the value that was viewed as noise or outliers may also contain an important pattern. Therefore, the remove noise section was temporarily disabled.

In addition, cross-validation with 10 folds was later added to improve the reliability of the evaluation result. By version 10, the dataset had collected enough (at least 120 sentences in total for each of the intruder and authenticated user) to produce more stable results. Additional experiments were carried out in version 12 to test noise removal again, but the performance continued to decrease. Therefore, the final workflow was completed in version 14.

The final baseline results are shown below:

MODEL	ACCURACY	PRECISION	RECALL	F1 SCORE
RANDOM FOREST	0.759180	0.759671	0.761801	0.760734
GRADIENT BOOSTING	0.722461	0.715831	0.742483	0.728913
KNN	0.671181	0.670195	0.680636	0.675375
SVM	0.612151	0.578418	0.841720	0.685660
LOGISTIC REGRESSION	0.586158	0.562923	0.789531	0.657243

4.2 Model Tuning

After evaluating the baseline models, Random Forest was selected as the main model for future tuning as it produced the highest performance in accuracy, precision, recall, and F1 score (Probst, Wright, & Boulesteix, 2019). The goal of the model tuning section was to optimise the model to improve predictive performance while maintaining robustness. The tuning process involved the following steps:

- **Pipeline Setup**

A Pipeline was created combining StandardScaler and RandomForestClassifier. Scaling was applied to normalise the features (hold_time, flight_time, and digraph_encoded), which improved model performance, particularly for algorithms sensitive to feature magnitudes.

- **Hyperparameter Grid**

Key Random Forest parameters were tuned using GridSearchCV with a stratified 10-fold cross-validation:

PARAMETER	VALUES TESTED
-----------	---------------

N_ESTIMATORS	100, 200, 300
MAX_DEPTH	None, 10, 20, 30
MIN_SAMPLES_SPLIT	2, 5, 10
MIN_SAMPLES_LEAF	1, 2, 4

- **Model Evaluation**

The tuned Random Forest was evaluated using 10-fold cross-validation, producing the following metrics:

METRIC	SCORE
ACCURACY	0.7705
PRECISION	0.7667
RECALL	0.7809
F1 SCORE	0.7738
AUC	0.8428

The confusion matrix indicated balanced classification between authenticated users and intruders, with a True Positive Rate (TPR) of 0.781 and a False Positive Rate (FPR) of 0.240.

- **Feature Importance**

Analysis of feature contributions revealed:

FEATURE	IMPORTANCE
HOLD TIME	37%
FLIGHT TIME	36.31%
DIGRAPH ENCODED	26.69%

This highlights that both temporal features and key transition patterns significantly impact authentication accuracy.

- **ROC Curve and AUC**

To assess the model's ability to distinguish classes, the ROC curve was plotted using cross-validated probability predictions. The AUC score was 0.843, indicating strong discriminative performance. The ROC visualisation confirms that the tuned Random Forest reliably separates authenticated users from intruders.

- **Feature Contribution Experiments**

To understand individual feature impact, the model was evaluated using single features:

- Hold Time only: Accuracy 66.02%
- Flight Time only: Accuracy 65.17%
- Digraph only: Accuracy 56.85%

This indicates that the digraph alone might not show a good pattern as all participants are tested using the same sentences, but the combination of digraph, hold and flight time can produce the best predictive performance.

- **Noise Removal Experiments**

After hyperparameter tuning, additional experiments were conducted to analyse the impact of removing extreme feature values. The tested values are as follows:

- Remove hold time that is greater than 0.5
- Remove hold time that is greater than 0.8
- Keep only flight time between -1 and 5
- Keep only flight time between -1 and 2
- Remove bad keys ('Shift_L', 'Caps_Lock', 'BackSpace')

In summary, hyperparameter tuning improved Random Forest accuracy and confirmed the importance of hold time, flight time, and encoded digraph. The ROC curve and AUC score reinforce the model's strong classification ability, setting the stage for comparison against other classifiers and further analysis in the next section.

4.3 Final Model

Based on the hyperparameter tuning in Section 4.2, the final Random Forest model was selected as the primary classifier for distinguishing authenticated users from intruders.

Final Model Configuration

The selected model uses the best parameters identified from GridSearchCV:

- `n_estimators`: 300
- `max_depth`: 30
- `min_samples_split`: 10
- `min_samples_leaf`: 1

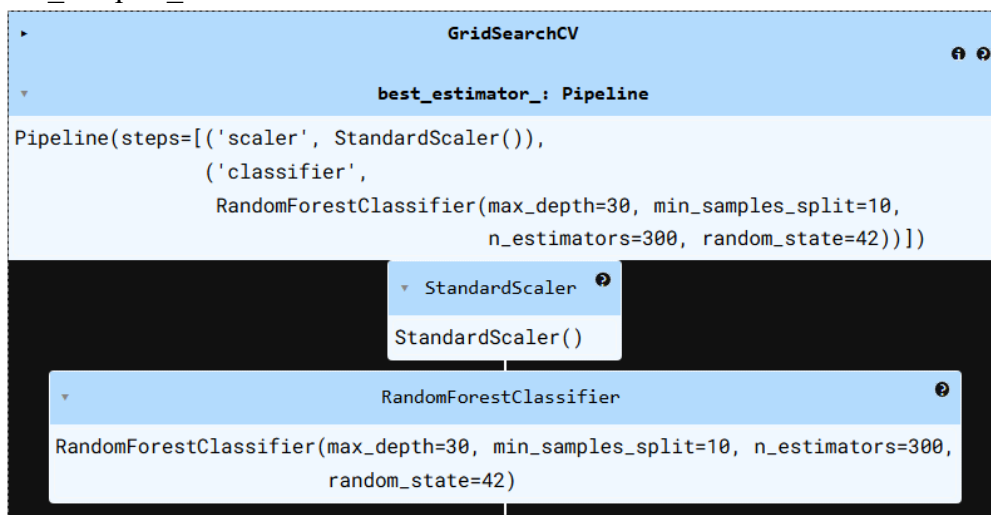


Figure 13: Visualise GridSearchCV result

The pipeline includes standard scaling of features (`hold_time`, `flight_time`, `digraph_encoded`) to ensure uniform feature distribution. This configuration achieved a cross-validation, improving slightly over the baseline Random Forest.

Noise Removal

After several tests, the only noise removal method that will increase the model's performance is removing the hold time that is greater than 0.8.

Confusion Matrix

The confusion matrix illustrates the model's classification performance:

- True Positives (authenticated correctly): 5009
- False Positives (intruder misclassified as authenticated): 1524
- True Negatives (intruder correctly identified): 4824
- False Negatives (authenticated misclassified as intruder): 1405

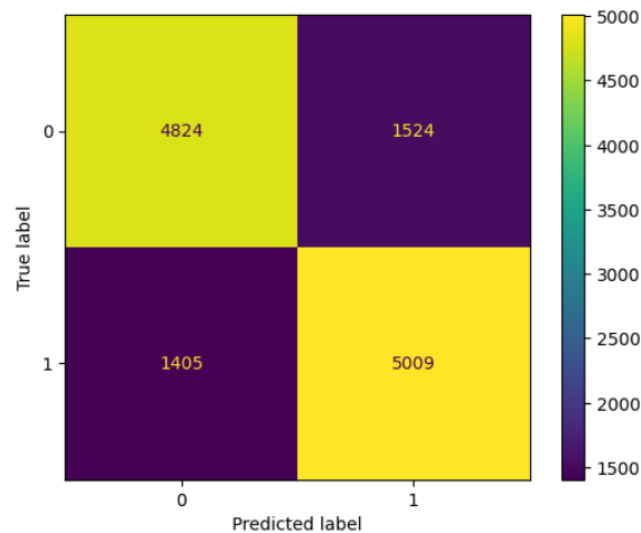


Figure 14: Final result confusion matrix

Feature Contribution

Feature importance analysis demonstrates the relative influence of each feature:

- Hold Time: 37%
- Flight Time: 36%
- Digraph Encoded: 27%

Single-feature experiments confirmed that combining all three features yields optimal accuracy. Temporal features (hold_time and flight_time) contribute the most, while digraph patterns provide additional discriminative power.

ROC Curve Visualisation

The ROC curve of the final model confirms its ability to separate classes effectively, with an AUC of 0.843. This high AUC value demonstrates that the final Random Forest reliably distinguishes authenticated users from intruders under varied typing patterns (Fawcett, 2006).

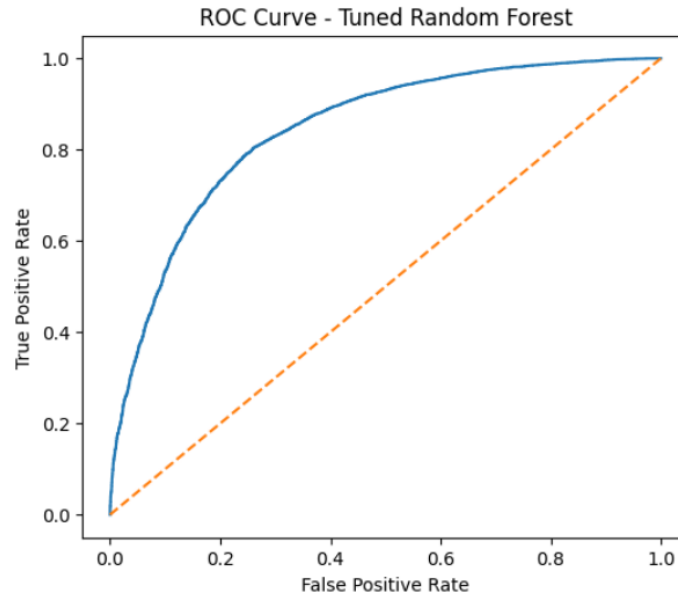


Figure 15: ROC Curve result plot

Summary

The final fine-tune model achieves 77.05% accuracy, compared to baseline 76.05% accuracy, a balanced trade-off between accuracy, precision, and recall, while leveraging all available features. It provides a strong basis for deploying behavioural keystroke authentication, with predictable performance as validated by cross-fold evaluation.

5. Results Analysis

This section shows the performance result of the machine learning models and provides an analysis of how effectively keystroke dynamics can be used for user authentication. The evaluation focuses on accuracy, precision, recall, F1 score, and AUC to ensure a comprehensive understanding of model performance.

5.1 Overall Model Performance

Among all the baseline models tested, Random Forest achieved the highest performance among all other models. This model also produces a higher accuracy of 0.7705 after tuning. This indicates that the system can correctly classify authenticated users and intruders in more than 77% of the cases. The precision and recall values were also relatively balanced, showing that the model performs consistently for both classes

The AUC score of 0.843 further confirms that the model is able to distinguish between authenticated users and intruders effectively. A high AUC value (0.843) also indicates strong classification ability even when the decision threshold changes (Fawcett, 2006).

5.2 Analysis of Keystroke Features

The results show that typing behaviour plays an important role in distinguishing users. Feature importance analysis revealed that hold time and flight time were the most significant features, while the digraph feature also contributed to the model performance.

Additional experiments were conducted using individual features to better understand their impact. When only hold time was used, the model achieved an accuracy of approximately 0.66, while using only flight time produced a similar result. However, using only digraph information resulted in much lower accuracy. This might indicate that timing behaviour is the most reliable attribute of user identity, while digraph patterns provide additional support when combined with the hold and flight timing features.

5.3 Effect of Data Quality and Noise

During the experiment, several attempts were made to remove outliers and noise data based on statistical values such as the mean and the 75th percentile. However, the results showed that removing them actually reduced the performance of the model. This suggests that extreme typing behaviour may still contain useful patterns that might help the model to differentiate between intruder and authenticated users.

In addition, the dataset became more stable after more participants were included. Earlier versions of the dataset produced inconsistent results, but performance improved once enough typing samples were collected.

5.4 Interpretation of the Results

The results demonstrate that keystroke dynamics can be effectively used as a behavioural biometric authentication method. Although the accuracy is not perfect, the model shows a strong ability to recognise typing patterns and differentiate between authorised users and intruders.

The study also highlights the importance of combining multiple features rather than relying on a single typing behaviour. The combination of hold time, flight time, and digraph patterns provides the most reliable results.

5.5 Additional Feature Filtering Experiments

Additional experiments were conducted before introducing the final result to further analyse how feature filtering affects model performance. Instead of modifying model parameters, this experiment focused on selectively removing specific data ranges and feature values to observe their impact on prediction accuracy.

The results showed that performance decreased when portions of the data were removed. For example, removing hold time values greater than 0.5 seconds reduced the overall accuracy. Similarly, restricting the flight time feature to only small ranges (such as -1 to 5 or -1 to 2) also resulted in lower performance. Removing certain “bad keys” or uncommon digraph patterns produced the same effect. This suggests that values that initially appear to be noise or outliers may still contain useful behavioural information that helps distinguish authenticated users from intruders.

However, a small improvement was observed when only extreme hold time values above 0.8 seconds were removed. Under this condition, the model accuracy increased slightly from 0.7690 to 0.7705. This indicates that extremely large hold time values may behave as true outliers, while moderate variations in typing behaviour still contribute positively to classification performance.

Overall, these experiments confirm that keystroke behavioural data is highly sensitive to feature removal. Instead of aggressively filtering the dataset, maintaining a wider range of typing behaviour generally produces better model performance. Therefore, only minimal filtering was applied in the final model.

6. Problems Encountered

During the development of the dynamic keylogging program, several technical and practical challenges were encountered.

6.1 Keylogger Compatibility Issue

Initially, a keylogger script obtained from GitHub was used because it can run in the background and capture keystroke data automatically. However, the keylogger could not function properly together with the text typing window used in this project, as the code can only run one of the features (if the GUI opens, the keyboard is not recorded; if the keyboard is recording, the GUI cannot open)

To solve this issue, the Python Tkinter built-in event feature was used instead of importing the “keyboard” library. The typing window was integrated directly into the program so that the key press and key release events could be recorded accurately in real time. This approach also ensures that the keystroke record will only run in the tkinter window, not in the background, which ensures ethical research purposes with explicit user consent, not malicious spyware that executes without user knowledge.

6.2 User Fatigue During Data Collection

Another challenge was related to user experience. Participants were required to type at least 20 sentences in order to collect enough keystroke data for training the machine learning model. However, typing a large number of sentences repeatedly can be tiring and boring, which may reduce user motivation, which could also affect typing behaviour and data consistency.

To improve the user experience, background music was initially planned. However, the feature was later decided not to include because, based on research done by Bramwell-Dicks, Petrie, & Edwards (2016), background music might influence the typing rhythm, which might affect the model performance. Instead, only some simple sound effects found on the “Myinstants” website were added to provide feedback with minimal effect on the typing behaviour.

6.3 Executable File Blocked by System Security

Another major issue occurred when the application was converted into a single executable (.exe) file and shared with other users for testing. Many systems, especially Windows computers with strong antivirus software, automatically block or delete the application because it was identified as a potential malware. In addition, macOS users were unable to even open the executable file, which might be due to system security restrictions or compatibility issues.

As a result, data collection had to be performed only on trusted devices where the program could run without being blocked. This limitation reduced the number of participants that could be included in the experiment.



Figure 16: Window security pop-up when executing the program

7. Testing and Quality Assurance

The testing and quality assurance section is to ensure that both the program implementation and the machine learning model function correctly. Since the project focuses on keystroke dynamics and behavioural authentication, testing was performed in three main areas: code functionality, baseline model validation, and final model evaluation.

7.1 Code and Data Collection Testing

The first stage of testing focused on verifying that the dynamic keylogging program workflow is correct and correctly collects and stores keystroke data. The typing interface was run and tested multiple times to ensure that the window pop-up sequence and key events were recorded accurately.

Additionally, some testing on the save feature is done by checking the saving file path and manually opening and observing the CSV file, to ensure that the collected values were consistent across different runs without any missing data. This step is important because any error in the data will directly affect the machine learning result.

7.2 Baseline Model Testing

After the dataset was created, baseline models were implemented and tested to verify that the machine learning pipeline worked correctly. Different models, such as K-Nearest Neighbour (KNN), Logistic Regression, Support Vector Machine (SVM), Gradient Boosting, and Random Forest, were trained using the correct dataset and evaluation method.

This testing stage ensured that the data preprocessing steps, label encoding, and training process were functioning properly. We can visualise the data using the Python print feature for a quick manual check on the output. Multiple versions saved and compared also ensure that the results were not produced randomly, as different models generated different accuracy levels.

7.3 Model Testing Using Cross-Validation

To improve the reliability of the results, 10-fold cross-validation was used. Instead of relying on a single training and testing split, the dataset was divided into ten subsets and the model was tested multiple times using different combinations of data, and the final result will be the average of these 10 cycles.

This ensures that the model maintains stable performance across different folds, which indicates that it is learning consistent typing patterns rather than memorising the training data, which improves the reliability of the evaluation results and reduces the risk of overfitting.

7.4 Performance Evaluation and Quality Assurance

Several evaluation metrics were used to ensure that the model's performance is strong and not based on random guessing. The final tuned Random Forest model achieved the following results:

- Accuracy: 0.7705
- Precision: 0.7667
- Recall: 0.7809
- F1 Score: 0.7738
- AUC Score: 0.8428

Precision and recall were used to verify that the model can correctly distinguish authenticated users from intruders, instead of random guessing or memorising. The ROC curve and AUC score further confirm that the model has strong classification ability, as an AUC value above 0.8 generally indicates good performance (Fawcett, 2006).

7.5 Verification of Improvement from Baseline to Tuned Model

To ensure that model tuning was meaningful, the performance of the tuned Random Forest model was compared with the baseline version. The results show a clear improvement in accuracy and F1 score after hyperparameter tuning and feature analysis.

This comparison confirms that the improvements were not caused by luck, but were the result of systematic testing and optimisation during the development process.

8. Limitation

Although the project successfully demonstrates the use of keystroke dynamics for behavioural authentication, several limitations should be considered when interpreting the results.

8.1 Limited Number of Participants and Data Size

One of the main limitations of this project is the relatively small dataset used to train and test the model. The dataset was created using around 120 sentences for the authenticated user and also 120 sentences for the intruder. While this amount of data might be enough to show the effectiveness of machine learning in keystroke authentication, it still may not be suitable to represent real-world typing behaviour.

In real-world keystroke authentication systems, the model should be trained with a much larger number of users or data. A larger dataset would allow the model to learn more diverse typing patterns and improve its ability to identify the authenticated user or intruder.

8.2 Use of Fixed Sentences

Another limitation of the project is the use of a fixed list of sentences during the data collection. All participants, including the authenticated user, were required to type the same set of sentences to ensure consistency in the dataset. This approach helps improve model performance because the digraph pattern remains similar across all typing sessions.

However, in a real-world environment, users do not type fixed sentences. Instead, they type different words, phrases, and sentences depending on the situation. This means that the typing patterns may vary, and the model may need to handle a much larger number of digraph combinations. As a result, the performance of the model in this project may be higher than what would be achieved in a real-world typing environment.

```
# Sentences used for typing test
sentences = [
    "Cybersecurity is crucial in the digital age.",
    "Artificial intelligence is transforming the world.",
    "Data privacy should be respected at all times.",
    "Machine learning models require quality data.",
    "Typing rhythm can reveal user identity.",
    "Biometric systems enhance digital security.",
    "Technology evolves faster than ever before.",
    "Strong passwords protect sensitive information.",
    "Software development requires logical thinking.",
    "User behavior can improve authentication systems.",
]
```

Figure 17: Overview of 10 predefined sentences in the list

8.3 Limited Feature Scope

The current model of this project focuses mainly on these three attributes, including hold time, flight time and encoded digraph. However, other behavioural features such as typing speed, typo rate, key pressure (if supported by hardware like a magnetic switch keyboard) or even mouse movement were not included.

Including more behavioural features could potentially improve the accuracy and robustness of the model. Therefore, the current program should be considered more like a simplified idea rather than a complete real-world authentication solution.

8.4 Platform and Data Collection Constraints

Another limitation is related to data collection and system compatibility. The program was mainly tested on a limited number of devices due to security restrictions when sharing the executable file. Some systems blocked or removed the program, which might be caused by an unknown publisher or keylogging feature being tagged as malicious, which restricted the number of participants that could be included in the experiment.

8.5 Limitations of the Machine Learning Model

This project mainly uses traditional machine learning classifiers such as Random Forest, Logistic Regression, K-Nearest Neighbour, Support Vector Machine, and Gradient Boosting. While these models are effective for smaller datasets and simpler feature sets in this project, they may not fully capture complex typing patterns over time.

More advanced models, especially deep learning approaches, may perform better in keystroke authentication tasks because they can automatically learn temporal patterns from typing sequences. For example, a Long Short-Term Memory (LSTM) network could be used to analyse typing behaviour as a time-series sequence rather than relying only on manually extracted features such as hold time and flight time. Similarly, models such as Recurrent Neural Networks (RNN) or deep neural networks could potentially improve accuracy if a larger dataset is available.

Conclusion

This project successfully developed a dynamic keylogging program and applied machine learning techniques to differentiate between different authenticated users and intruders based on keystroke dynamics. Using Python and the tkinter library, a controlled typing environment was created to collect hold time, flight time, and digraph features. The collected dataset, though limited in size, allowed the development and evaluation of multiple classification models, with Random Forest achieving the highest performance. After tuning, the final model reached 77.05% accuracy, balanced precision and recall, and an AUC of 0.843, indicating strong discriminative capability.

The results demonstrate that keystroke dynamics can serve as a viable behavioural biometric for authentication, particularly when combining multiple features rather than relying on a single attribute. However, limitations such as a fixed set of sentences, a relatively small number of participants, and platform-specific data collection constraints suggest that real-world deployment may require further data diversity and testing across broader typing scenarios.

Overall, the project highlights the potential of behavioural keystroke analysis for secure authentication, while providing a practical workflow for integrating data collection, machine learning, and evaluation in a reproducible manner.

Reference

- boppreh/keyboard: Hook and simulate global keyboard events on Windows and Linux. (n.d.). GitHub. <https://github.com/boppreh/keyboard#api>
- Firdose, T. (2023, August 24). Fine-tuning your random forest classifier: A guide to hyperparameter tuning. Medium. <https://tahera-firdose.medium.com/fine-tuning-your-random-forest-classifier-a-guide-to-hyperparameter-tuning-d5ceab0c4852>
- Bramwell-Dicks, Anna & Petrie, Helen & Edwards, Alistair. (2016). Can Listening to Music Make You Type Better? The Effect of Music Style, Vocals and Volume on Typing Performance. [109-116](#). [10.21785/icad2016.029](#)
- Fawcett, Tom. (2006). Introduction to ROC analysis. Pattern Recognition Letters. 27. 861-874. [10.1016/j.patrec.2005.10.010](#)
- Oliveira, S. V. B., Silva, T. H. S., Saraiva, P. A. R., Souza, E. F., Godoy, R. V., Ambrosio, L. A., & Becker, M. (2025). The Impact of Feature Scaling In Machine Learning. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2025.3635541>
- Probst, P., Wright, M., & Boulesteix, A. L. (2019). Hyperparameters and Tuning Strategies for Random Forest. *arXiv preprint arXiv:1804.03515*. <https://arxiv.org/abs/1804.03515>
- Wahab, A. A., Manno, M., Lukaszewski, M., Hou, D., & Hussain, F. (2023). Keystroke Dynamics: Concepts, Techniques, and Applications. *arXiv preprint arXiv:2303.04605*. <https://arxiv.org/abs/2303.04605>

Sound effect

- Myinstants. (n.d.). Correct answer - Sound Button. Myinstants. <https://www.myinstants.com/en/instant/correct-answer/>
- Myinstants. (n.d.). Duolingo completed lesson - Sound Button. Myinstants. <https://www.myinstants.com/en/instant/duolingo-completed-lesson-48481/>
- Myinstants. (n.d.). Typewriter - Sound button. Myinstants. <https://www.myinstants.com/en/instant/typewriter-15339/>
- Myinstants. (n.d.). Windows 7 error sound - Sound Button. Myinstants. <https://www.myinstants.com/en/instant/windows-7-error-sound-73457/>

Appendix:

Python code:

- The source code will be zipped and attached to the submission box.
- The single executable file will be located in CP3416-Keylogger-Project\dist\keylogger.exe

Kaggle (Notebook):

- Kaggle (baseline with 5 models): <https://www.kaggle.com/code/limdx006/keylogger-project-baseline>
- Kaggle (Random Forest): <https://www.kaggle.com/code/limdx006/keylogger-project-random-forest>

Kaggle (Data):

- <https://www.kaggle.com/datasets/limdx006/authenticated-user-keylog>